

Министерство образования и науки федеральное государственное
автономное образовательное учреждение высшего образования «Национальный
исследовательский университет ИТМО»

Факультет прикладной информатики

Курсовая работа

**Тема: Разработка Android-приложения с использованием
архитектуры MVVM, REST API, локального хранилища Room и фоновой
синхронизации**

Выполнила:

Отрощенко Валерия Александровна

Проверили:

Шатинский Григорий Сергеевич

Санкт-Петербург

2025 г.

ВВЕДЕНИЕ.....	3
1 Проектирование приложения.....	5
1.1 Общая архитектура приложения.....	5
1.1.1 Уровень представления (View).....	6
1.1.2 Уровень модели представления (ViewModel).....	7
1.1.3 Уровень модели данных (Model).....	8
1.1.3.1 Работа с сетью.....	8
1.1.3.2 Локальное хранилище данных.....	8
1.1.3.3 Repository как единая точка доступа.....	8
1.1.4 Вспомогательные и фоновые компоненты.....	9
1.2 Проектирование навигации и экранов приложения.....	9
1.2.1 Навигационная модель приложения.....	9
1.2.2 Описание экранов приложения.....	11
1.2.2.1 Экран ленты сообщений.....	11
1.2.2.2 Экран профиля пользователя.....	12
1.2.2.3 Экран настроек приложения.....	13
1.2.3 Связь экранов с архитектурой MVVM.....	13
2 Реализация приложения.....	14
2.1 Реализация пользовательского интерфейса и навигации.....	14
2.1.1 Структура главного экрана и контейнер навигации.....	14
2.1.2 Навигационный граф приложения.....	15
2.1.3 Нижнее меню навигации.....	16
2.1.4 Инициализация навигации в MainActivity.....	17
3 Интеграция сетевых, локальных и фоновых механизмов обработки данных.....	18
3.1 Архитектура слоя данных и паттерн Repository.....	18
3.2 Поддержка офлайн-режима и обработка сетевых ошибок.....	19
3.3 Фоновая синхронизация данных с использованием WorkManager.....	19
3.4 Реализация системы уведомлений.....	20
3.5 Итоговый сценарий работы приложения.....	21
ЗАКЛЮЧЕНИЕ.....	23

ВВЕДЕНИЕ

Современные мобильные приложения являются ключевым элементом цифровых экосистем, обеспечивая пользователям доступ к информации и сервисам в режиме реального времени. Особое значение имеют приложения, работающие с удалёнными источниками данных, поддерживающие офлайн-доступ и устойчивые к изменениям состояния устройства, таким как поворот экрана или временное отсутствие сети.

В процессе разработки Android-приложений особое внимание уделяется архитектуре приложения, так как от неё зависят масштабируемость, поддерживаемость и устойчивость системы. Использование устаревших архитектурных подходов, при которых логика приложения сосредоточена в Activity или Fragment, приводит к усложнению кода и повышает вероятность ошибок.

Целью данной курсовой работы является разработка Android-приложения, реализующего:

- 1) современную архитектуру MVVM;
- 2) получение данных из сетевого REST API;
- 3) сохранение данных в локальной базе данных;
- 4) поддержку офлайн-режима;
- 5) фоновую синхронизацию данных;
- 6) уведомление пользователя о результатах синхронизации.

Для достижения поставленной цели были определены следующие задачи:

- 1) разработать пользовательский интерфейс с использованием компонентов Material Design;
- 2) реализовать управление состоянием экранов с помощью ViewModel;
- 3) интегрировать сетевое взаимодействие с использованием Retrofit и корутин;

- 4) организовать локальное хранилище данных на базе Room;
- 5) обеспечить фоновое обновление данных с использованием WorkManager;
- 6) реализовать механизм уведомлений и запрос системных разрешений.

1 Проектирование приложения

1.1 Общая архитектура приложения

Проектируемое Android-приложение реализовано с использованием архитектурного паттерна MVVM (Model–View–ViewModel), который обеспечивает четкое разделение ответственности между компонентами пользовательского интерфейса, логикой управления состоянием и слоем данных. Выбор данной архитектуры обусловлен требованиями устойчивости приложения к изменениям конфигурации устройства, поддержкой оффлайн-режима и необходимостью масштабируемости программного решения.

Архитектура приложения организована в виде набора логических пакетов, каждый из которых соответствует определенному уровню MVVM (см. рисунок 1). Такое разделение повышает читаемость кода, упрощает сопровождение и позволяет изолировать изменения в рамках одного слоя без влияния на остальные компоненты системы.

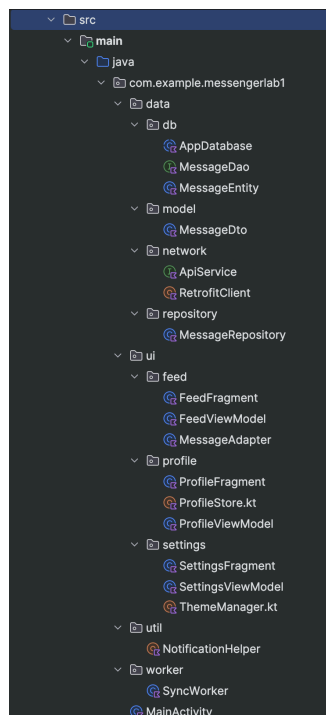


Рисунок 1 – Структура проекта

1.1.1 Уровень представления (View)

К уровню представления относятся классы, расположенные в пакете `ui`, а также главный класс `MainActivity`. Данный уровень отвечает за отображение пользовательского интерфейса, обработку пользовательских действий и взаимодействие с системой навигации.

`MainActivity` выполняет роль контейнера пользовательского интерфейса и отвечает за инициализацию навигации с использованием `NavHostFragment`, настройку нижнего меню `BottomNavigationView`, управление отображением `FloatingActionButton`, обработку системных разрешений и инициализацию фоновых задач и каналов уведомлений.

Каждый экран приложения реализован в виде отдельного `Fragment`, сгруппированного по функциональному признаку:

- 1) `FeedFragment`: отображение ленты сообщений;
- 2) `ProfileFragment`: работа с профилем пользователя;
- 3) `SettingsFragment`: управление настройками приложения.

Компоненты уровня `View` не содержат бизнес-логики и не взаимодействуют напрямую с источниками данных. Все данные передаются в пользовательский интерфейс через соответствующие классы `ViewModel`.

1.1.2 Уровень модели представления (ViewModel)

Уровень `ViewModel` представлен классами `FeedViewModel`, `ProfileViewModel` и `SettingsViewModel`, расположенными в соответствующих пакетах внутри `ui`. `ViewModel` выполняют роль посредника между пользовательским интерфейсом и слоем данных.

Основные функции `ViewModel`:

- 1) хранение состояния экранов;
- 2) выполнение асинхронных операций с использованием корутин;
- 3) обработка пользовательских действий;

- 4) предоставление данных в виде объектов LiveData.

Использование ViewModel позволяет сохранять состояние приложения при изменениях конфигурации устройства, таких как поворот экрана, без необходимости повторной загрузки данных. Это обеспечивает устойчивость приложения и улучшает пользовательский опыт.

1.1.3 Уровень модели данных (Model)

Слой модели данных реализован в пакете data и включает компоненты, отвечающие за получение, хранение и предоставление данных.

1.1.3.1 Работа с сетью

Взаимодействие с удалённым REST API осуществляется через пакет data.network, включающий:

- 1) ApiService – интерфейс Retrofit с описанием сетевых запросов;
- 2) RetrofitClient – объект для конфигурации сетевого клиента.

Сетевые операции выполняются асинхронно с использованием корутин, что предотвращает блокировку пользовательского интерфейса.

1.1.3.2 Локальное хранилище данных

Для обеспечения офлайн-доступа реализована локальная база данных Room, расположенная в пакете data.db. В неё входят:

- 1) AppDatabase – основной класс базы данных;
- 2) MessageDao – интерфейс доступа к данным;
- 3) MessageEntity – сущность, описывающая структуру таблицы сообщений.

Данные, полученные из сети, сохраняются в базе данных и используются при отсутствии сетевого соединения.

1.1.3.3 Repository как единая точка доступа

Класс `MessageRepository`, расположенный в пакете `data.repository`, реализует паттерн `Repository` и является единой точкой взаимодействия `ViewModel` с данными. `Repository` инкапсулирует логику выбора источника данных и обеспечивает:

- 1) загрузку данных из сети при наличии подключения;
- 2) сохранение данных в локальную базу;
- 3) возврат данных из базы при отсутствии сети.

1.1.4 Вспомогательные и фоновые компоненты

Для реализации фоновой синхронизации используется класс `SyncWorker`, расположенный в пакете `worker`. Он выполняет периодическое обновление данных с использованием `WorkManager` и работает независимо от пользовательского интерфейса.

Для отображения уведомлений реализован вспомогательный класс `NotificationHelper`, вынесенный в пакет `util`. Это позволяет изолировать логику работы с уведомлениями от бизнес-логики приложения.

1.2 Проектирование навигации и экранов приложения

Проектирование навигации и экранов приложения является важным этапом разработки, так как именно от него зависит удобство взаимодействия пользователя с системой и логичность пользовательских сценариев. В рамках данной курсовой работы навигационная модель приложения была спроектирована с учётом принципов пользовательского опыта (UX), рекомендаций платформы Android и выбранной архитектуры MVVM.

1.2.1 Навигационная модель приложения

В качестве основы навигации в приложении используется компонент `Jetpack Navigation`, который является частью `Android Architecture Components` и

рекомендован Google для построения навигации в современных Android-приложениях.

Навигационная модель приложения реализована следующим образом:

- 1) приложение содержит одну MainActivity, выступающую в роли контейнера пользовательского интерфейса;
- 2) внутри MainActivity размещён NavHostFragment, который управляет переключением экранов;
- 3) основные экраны приложения доступны через нижнюю навигационную панель BottomNavigationView.

Использование нижней навигации обусловлено тем, что приложение содержит несколько равнозначных разделов, между которыми пользователь может свободно переключаться в любом порядке. Данный тип навигации соответствует рекомендациям Material Design и широко применяется в мобильных приложениях с несколькими основными разделами.

В нижнюю навигационную панель включены следующие пункты (см. рисунок 2):

- 1) Лента – основной экран приложения, отображающий список сообщений;
- 2) Профиль – экран для просмотра и редактирования пользовательских данных;
- 3) Настройки – экран для управления параметрами приложения.

Каждый пункт нижней навигации связан с соответствующим Fragment, описанным в навигационном графе (nav_graph.xml). Использование навигационного графа позволяет централизованно управлять переходами между экранами и упрощает сопровождение кода.



Рисунок 2 – Навигация в приложении

1.2.2 Описание экранов приложения

В рамках приложения реализовано несколько экранов, каждый из которых отвечает за отдельную функциональную область. Такое разделение позволяет изолировать ответственность экранов и упростить дальнейшее расширение функциональности.

1.2.2.1 Экран ленты сообщений

Экран ленты сообщений является основным экраном приложения и реализован в виде `FeedFragment`. Он предназначен для отображения списка сообщений, полученных из сетевого API или локальной базы данных.

Основные элементы экрана:

- 1) список сообщений, реализованный с помощью RecyclerView;
- 2) карточки сообщений, оформленные с использованием MaterialCardView;
- 3) кнопка обновления (FloatingActionButton), позволяющая вручную инициировать загрузку данных.

Каждое сообщение в ленте содержит аватар пользователя, имя автора, текст сообщения, иконку «лайка», состояние которой может изменяться пользователем.

Экран ленты поддерживает как онлайн-, так и офлайн-режим работы. При отсутствии сетевого соединения данные загружаются из локальной базы данных Room.

1.2.2.2 Экран профиля пользователя

Экран профиля пользователя реализован в виде ProfileFragment и предназначен для отображения и редактирования персональной информации пользователя.

На экране профиля отображаются следующие данные:

- 1) имя пользователя;
- 2) никнейм;
- 3) адрес электронной почты;
- 4) номер телефона;
- 5) краткое описание («о себе»).

Редактирование данных осуществляется с использованием полей ввода, а состояние профиля сохраняется через ProfileStore. Для управления состоянием экрана используется ProfileViewModel, что обеспечивает сохранение введенных данных при повороте экрана и других изменениях конфигурации.

1.2.2.3 Экран настроек приложения

Экран настроек реализован в виде `SettingsFragment` и предназначен для управления параметрами приложения. В рамках курсовой работы основным элементом экрана настроек является переключатель темы приложения (светлая/тёмная).

Состояние выбранной темы сохраняется с использованием `ThemeManager` и предоставляется пользовательскому интерфейсу через `SettingsViewModel`. Это позволяет применять выбранную тему ко всему приложению и сохранять настройки между запусками.

1.2.3 Связь экранов с архитектурой MVVM

Каждый экран приложения интегрирован в архитектуру MVVM и имеет чётко определённое взаимодействие между слоями `View`, `ViewModel` и `Model`.

Связь компонентов реализована следующим образом:

- 1) `Fragment` отвечает исключительно за отображение данных и обработку пользовательского ввода;
- 2) `ViewModel` хранит состояние экрана и обрабатывает пользовательские действия;
- 3) слой модели данных предоставляет необходимые данные через `Repository` или специализированные хранилища.

Например, для экрана ленты сообщений:

- 1) `FeedFragment` отображает список сообщений и обрабатывает действия пользователя;
- 2) `FeedViewModel` хранит список сообщений в виде `LiveData` и инициирует обновление данных;
- 3) `MessageRepository` получает данные из сети или базы данных.

Такое разделение ответственности позволяет избежать дублирования логики, упростить тестирование отдельных компонентов, обеспечить

реактивное обновление пользовательского интерфейса и повысить устойчивость приложения к изменениям конфигурации устройства.

2 Реализация приложения

2.1 Реализация пользовательского интерфейса и навигации

2.1.1 Структура главного экрана и контейнер навигации

Главный экран приложения реализован в MainActivity, которая выступает контейнером для NavHostFragment и нижней навигационной панели. В качестве корневого layout используется CoordinatorLayout, обеспечивающий корректное взаимодействие элементов интерфейса (например, AppBar и прокручиваемого контента).

- 1) MaterialToolbar используется как верхняя панель приложения (AppBar);
- 2) NavHostFragment содержит активный фрагмент и управляет навигацией;
- 3) BottomNavigationView обеспечивает быстрый доступ к основным разделам приложения.

Использование `layout_marginBottom="56dp"` для контейнера навигации обеспечивает резерв пространства под нижнюю навигацию и предотвращает перекрытие контента (см. листинг 1).

Листинг 1 – Разметка `activity_main.xml` (контейнер навигации и нижнее меню)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    tools:context=".MainActivity">
```

```

<com.google.android.material.appbar.AppBarLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:fitsSystemWindows="true">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/topAppBar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:fitsSystemWindows="true"
        app:layout_scrollFlags="scroll|enterAlways"/>
</com.google.android.material.appbar.AppBarLayout>

<fragment
    android:id="@+id/nav_host_fragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginBottom="56dp"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph"
    app:layout_behavior="@string/appbar_scrolling_view_behavior" />

<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    app:menu="@menu/bottom_nav_menu"
    app:labelVisibilityMode="labeled" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

2.1.2 Навигационный граф приложения

Переходы между экранами определены в навигационном графе `nav_graph.xml`. В граф включены основные фрагменты приложения: лента сообщений, профиль и настройки. Навигационный граф служит единым источником правды для системы навигации (см. листинг 2).

Листинг 2 – Фрагмент nav_graph.xml (описание экранов приложения)

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/feedFragment">

    <fragment
        android:id="@+id/feedFragment"
        android:name="com.example.messengerlab1.ui.feed.FeedFragment"
        android:label="Лента" />

    <fragment
        android:id="@+id/profileFragment"
        android:name="com.example.messengerlab1.ui.profile.ProfileFragment"
        android:label="Профиль" />

    <fragment
        android:id="@+id/settingsFragment"
        android:name="com.example.messengerlab1.ui.settings.SettingsFragment"
        android:label="Настройки" />

</navigation>
```

2.1.3 Нижнее меню навигации

Нижняя навигация реализована через BottomNavigationView, элементы которой задаются отдельным XML-файлом меню. Каждый пункт меню связан с соответствующим экраном навигационного графа по идентификатору (см. листинг 3).

Листинг 3 – Фрагмент bottom_nav_menu.xml (элементы нижней навигации)

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
```

```

        android:id="@+id/feedFragment"
        android:title="Лента"
        android:icon="@android:drawable/ic_menu_view" />

        <item
            android:id="@+id/profileFragment"
            android:title="Профиль"
            android:icon="@android:drawable/ic_menu_myplaces" />

        <item
            android:id="@+id/settingsFragment"
            android:title="Настройки"
            android:icon="@android:drawable/ic_menu_preferences" />

    </menu>

```

2.1.4 Инициализация навигации в MainActivity

Связь BottomNavigationView и навигационного контроллера выполняется в MainActivity (см. листинг 4). Это обеспечивает корректное переключение экранов по нажатию на пункты меню и поддержку навигации «назад».

Листинг 4 – Фрагмент MainActivity.kt (инициализация навигации)

```

val navHostFragment =
    supportFragmentManager.findFragmentById(R.id.nav_host_fragment) as
NavHostFragment

val navController = navHostFragment.navController
binding.bottomNav.setupWithNavController(navController)

```

Данный фрагмент обеспечивает получение NavController из NavHostFragment и подключение нижней навигации к NavController с помощью setupWithNavController.

3 Интеграция сетевых, локальных и фоновых механизмов обработки данных

3.1 Архитектура слоя данных и паттерн Repository

Центральным элементом слоя данных приложения является класс MessageRepository, реализующий паттерн Repository. Repository служит единой

точкой доступа к данным для ViewModel и инкапсулирует логику работы с различными источниками данных (см. листинг 5).

Использование Repository позволяет:

- 1) изолировать ViewModel от деталей реализации сетевого и локального хранения;
- 2) централизовать обработку ошибок;
- 3) упростить изменение источников данных без модификации UI-слоя.

Repository взаимодействует с двумя основными источниками данных: удаленным REST API (через Retrofit) и локальной базой данных Room.

Листинг 5 – Фрагмент реализации MessageRepository

```
class MessageRepository(  
    private val api: ApiService,  
    private val dao: MessageDao  
) {  
  
    suspend fun getMessages(forceRefresh: Boolean): List<MessageEntity> {  
        return if (forceRefresh) {  
            refreshFromNetworkOrFallback()  
        } else {  
            dao.getAll()  
        }  
    }  
  
    suspend fun refreshFromNetworkOrFallback(): List<MessageEntity> {  
        return try {  
            val remote = api.getMessages()  
            val entities = remote.map {  
                MessageEntity(  
                    id = it.id,  
                    title = it.title,  
                    body = it.body,  
                    liked = false  
                )  
            }  
            dao.insertAll(entities)  
            entities  
        } catch (e: Exception) {
```

```

        dao.getAll()
    }
}
}

```

3.2 Поддержка офлайн-режима и обработка сетевых ошибок

Поддержка офлайн-режима является важным требованием для мобильных приложений, ориентированных на работу с сетевыми данными. В разработанном приложении офлайн-режим реализован на основе локальной базы данных Room.

При наличии интернет-соединения данные загружаются из сети и сохраняются в базе данных. При отсутствии сети Repository автоматически возвращает данные из локального хранилища, что позволяет пользователю продолжать работу с приложением без сбоев.

Такой подход соответствует концепции *offline-first*, при которой локальные данные рассматриваются как основной источник информации, а сеть – как механизм синхронизации.

Использование конструкции *try-catch* в Repository позволяет корректно обрабатывать сетевые ошибки без аварийного завершения приложения. Пользовательский интерфейс при этом продолжает функционировать и отображает ранее загруженные данные.

3.3 Фоновая синхронизация данных с использованием WorkManager

Для автоматического обновления данных без участия пользователя в приложении реализована фоновая синхронизация с использованием компонента WorkManager. Данный механизм гарантирует выполнение фоновых задач даже в случае перезапуска приложения или устройства.

В рамках курсовой работы реализован класс SyncWorker, наследующий CoroutineWorker, что позволяет выполнять сетевые операции асинхронно с использованием корутин.

Листинг 6 – Реализация фонового Worker

```

class SyncWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        val repo = ServiceLocator.repository
        repo.refreshFromNetworkOrFallback()
        NotificationHelper.showSyncSuccess(applicationContext)
        return Result.success()
    }
}

```

Листинг 7 – Планирование периодической синхронизации

```

val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .build()

val request = PeriodicWorkRequestBuilder<SyncWorker>(15, TimeUnit.MINUTES)
    .setConstraints(constraints)
    .build()

WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    "sync_messages",
    ExistingPeriodicWorkPolicy.UPDATE,
    request
)

```

3.4 Реализация системы уведомлений

Для информирования пользователя о результатах фоновой синхронизации в приложении реализована система уведомлений. При успешном обновлении данных пользователю отображается уведомление с текстом «Новые данные получены».

Для этого используется вспомогательный класс `NotificationHelper`, отвечающий за создание канала уведомлений и отображение уведомлений.

Листинг 8 – Фрагмент `NotificationHelper`

```

object NotificationHelper {

```

```

fun ensureChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            "sync",
            "Data sync",
            NotificationManager.IMPORTANCE_DEFAULT
        )

        val manager =
context.getSystemService(NotificationManager::class.java)
        manager.createNotificationChannel(channel)
    }
}

fun showSyncSuccess(context: Context) {
    val notification = NotificationCompat.Builder(context, "sync")
        .setSmallIcon(android.R.drawable.ic_popup_sync)
        .setContentTitle("Синхронизация")
        .setContentText("Новые данные получены")
        .build()

    NotificationManagerCompat.from(context).notify(1, notification)
}
}

```

3.5 Итоговый сценарий работы приложения

Реализованная архитектура обеспечивает следующий сквозной сценарий работы приложения:

- 1) Пользователь запускает приложение.
- 2) Данные загружаются из локальной базы данных или из сети.
- 3) Пользователь взаимодействует с лентой сообщений, включая возможность «лайкать» сообщения.
- 4) При временном отсутствии интернет-соединения приложение продолжает работать, используя локальные данные.
- 5) В фоновом режиме периодически запускается синхронизация данных.

6) При успешном обновлении данных пользователь получает системное уведомление.

7) Состояние приложения сохраняется при повороте экрана и других изменениях конфигурации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана Android-система, реализующая современные подходы к проектированию и разработке мобильных приложений. Основной целью работы являлось создание приложения, обеспечивающего получение данных из сетевого источника, их сохранение в локальной базе данных, поддержку офлайн-режима, а также фоновую синхронизацию и уведомление пользователя о результатах обновления данных. Поставленная цель была полностью достигнута.

В процессе выполнения работы были последовательно решены все поставленные задачи. На этапе проектирования была выбрана архитектура MVVM, позволяющая чётко разделить ответственность между пользовательским интерфейсом, логикой управления состоянием и слоем данных. Применение данной архитектуры обеспечило устойчивость приложения к изменениям конфигурации устройства, улучшило читаемость кода и упростило дальнейшее расширение функциональности.

В рамках реализации пользовательского интерфейса был создан каркас приложения с использованием компонентов Material Design и Jetpack Navigation. Были реализованы основные экраны приложения, включая ленту сообщений, профиль пользователя и экран настроек. Навигационная модель приложения обеспечивает интуитивно понятное взаимодействие пользователя с системой и соответствует рекомендациям платформы Android.

Для управления состоянием экранов были использованы классы ViewModel и механизм LiveData, что позволило реализовать реактивное обновление пользовательского интерфейса и сохранить данные при повороте экрана и других изменениях конфигурации. Данный подход исключил необходимость повторной загрузки данных и повысил производительность приложения.

В ходе реализации работы с данными была выполнена интеграция сетевого взаимодействия с использованием библиотеки Retrofit и корутин. Для обеспечения офлайн-доступа и устойчивости к сетевым сбоям была реализована локальная база данных Room. Использование паттерна Repository позволило централизовать логику работы с источниками данных и реализовать стратегию offline-first.

Значительным этапом работы стала реализация фоновой синхронизации данных с использованием WorkManager. Данный механизм обеспечивает автоматическое обновление данных в фоновом режиме с учётом системных ограничений и требований энергоэффективности. Для повышения информативности пользовательского опыта была реализована система уведомлений, информирующая пользователя о результатах синхронизации данных.

Таким образом, в рамках курсовой работы было разработано функционально завершённое мобильное приложение, демонстрирующее комплексный подход к работе с пользовательским интерфейсом, состоянием, данными и фоновыми процессами. Полученные результаты подтверждают целесообразность использования современных архитектурных решений и инструментов Android-разработки.

